Brief Announcement: Computability and Anonymous Storage-Efficient Consensus with an Abstract MAC Layer

Lewis Tseng
lewis.tseng@bc.edu
Boston College
Massachusetts, USA

Qinzi Zhang
qinziz@bu.edu
Boston University
Massachusetts, USA

ABSTRACT

This paper explores fault-tolerant algorithms in the abstract MAC layer [7] in a single-hop network. The model captures the basic properties of modern wireless MAC protocols. Newport [11] proves that it is impossible to achieve deterministic fault-tolerant consensus, and Newport and Robinson [10] present randomized crash-tolerant consensus algorithms. We are not aware of any study on the computability in this model.

This paper presents a straightforward construction of the *store-collect* object, which then allows us to apply prior algorithms to solve many well-known distributed problems, such as multi-writer atomic registers, counters, atomic snapshot objects, and approximate and randomized consensus. Our construction does not require a priori information on the participating nodes. Our insight also leads to *anonymous* approximate consensus and randomized consensus algorithms that use a constant number of variables (or values). All of the algorithms are *wait-free*.

CCS CONCEPTS

• **Theory of computation → Distributed algorithms**.

KEYWORDS

Abstract MAC Layer, Computation Power, Consensus

ACM Reference Format:
Lewis Tseng and Qinzi Zhang. 2022. Brief Announcement: Computability and Anonymous Storage-Efficient Consensus with an Abstract MAC Layer. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (PODC '22), July 25–29, 2022, Salerno, Italy*. ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3519270.3538462

1 INTRODUCTION

Connected devices and computing entities communicating over wireless networks are growing at an unprecedented pace. International Data Corporation (IDC) predicts that by 2025 there will be 55.7 B connected devices, together generating 73.1 ZB amount of data. These devices can be used in a wide range of applications such as Industrial Internet-of-Things, wearable devices, sensor networks, smart city, etc. Most connected devices are small and fragile; hence, it is necessary to design fault-tolerant algorithms for practical reasons. On the other hand, standard message-passing models [5, 6, 9] focus on point-to-point communication channels. Therefore, from a theoretical perspective, it is important to understand whether the broadcast and anonymous nature of the wireless communication medium affect computability.

In this paper, we are interested in the fault-tolerant primitives, especially with the focus on the aspect of wireless links in a single-hop network. In particular, we adopt the *Abstract MAC layer* model [7, 8, 10, 11] – if a fault-free node i broadcasts a message m, then m will eventually be delivered to all the nodes that have not crashed yet, including i itself, and upon the broadcast is completed, node i receives an acknowledgment of the completion of the broadcast. No other information is provided by the abstract MAC layer. For example, node i does not learn the network size n, nor the identity of receiving nodes. The computation and communication are assumed to be *asynchronous*.

The model captures the basic properties of existing wireless MAC (medium access control) layers such as TDMA (time-division multiple access) or CSMA (carrier-sense multiple access). Even though the abstraction does not model after any specific existing MAC layer, the abstract MAC layer still serves an important goal – the separation of high-level algorithm design and low-level logic of handling wireless channel and managing participating nodes. As argued in [10, 11], this separation allows us to identify principles that can hopefully help fill the gap between theory and practice in designing algorithms that can be readily deployed onto MAC layers that are common in modern wireless devices. In fact, recent works propose approaches to implement the abstract MAC layer in realistic network conditions [12, 13].

It is important to study computability of a given model, which gives us insights on the solvability of distributed computation tasks. Newport [11] proves that deterministic consensus is impossible in the abstract MAC layer when nodes may crash. Subsequently, Newport and Robinson [10] propose randomized consensus algorithms. We are not aware of any results on other fault-tolerant primitives. We show that the abstract MAC layer is fundamentally related to the *store-collect object* [3, 4] by presenting a simple algorithm to implement the object using the abstract MAC layer. Then "stacking" the constructions in [4] on top of our store-collect object solves many well-known computation tasks, e.g., multi-writer registers, counters, atomic snapshot objects, and approximate, and randomized consensus.

An important implication of our result addresses a claim in [10] – it was stated that "*register simulations are also not an option in our model (the abstract MAC layer)*." It is true that the standard simulation techniques, e.g., [2], that require the knowledge of network size and quorum are not feasible; however, as shown in our

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
PODC '22, July 25–29, 2022, Salerno, Italy
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9262-4/22/07.
https://doi.org/10.1145/3519270.3538462

construction, the broadcast primitive and the acknowledgement provided by the abstract MAC layer model are indeed sufficient for building atomic objects.

2 PRELIMINARY

Models: We consider a static system consisting of n nodes. That is, we do not consider node churn. Each node is assumed to have a unique identifier; hence, the set of nodes is also denoted as the set of their identifiers, i.e., $\{1, \ldots, n\}$. The nodes are assumed to be asynchronous, which may proceed at an arbitrary speed. We consider the crash fault model in which any number of nodes may fail. A faulty node may crash and stop execution at any point of time. The adversary may control the faulty behavior, the set of faulty nodes, and message delays. Nodes that are not faulty are called fault-free nodes.

In the abstract MAC layer model [7, 8, 10, 11], nodes communicate using a `mac-broadcast` primitive provided by the underlying layer. The primitive eventually delivers the message to all the nodes that have not crashed yet, including i itself. Moreover, at some point after the `mac-broadcast` has succeeded in delivering the broadcast message, the broadcaster receives an acknowledgement, representing that the `mac-broadcast` is complete. Note that the broadcaster *cannot* infer any other information from the acknowledgment, not the network size n, nor the identities of the receivers. A crash may occur during the `mac-broadcast`, which results into inconsistency. If a broadcaster crashes in the middle of the process, then some nodes might receive the message while others do not. We assume a single-hop network.

Algorithm Presentation and Message Processing: Each node is assumed to take steps sequentially. Each line of the pseudo-code is executed *atomically*, except when calling `mac-broadcast`, since this primitive is handled by the underlying abstract MAC layer. We present the algorithm using an event-driven approach. When a certain event occurs (e.g., a Store operation is invoked), then a block of code is being executed. Additionally, each algorithm also has a message handler that processes incoming messages. Our algorithms assume that (i) the message handler is triggered whenever the underlying layer receives a message and sends an interrupt; (ii) there is only one message handler thread, which processes messages one by one (i.e., the underlying layer has a queue of pending messages); and (iii) the handler has a priority over the execution of the main thread.

Note that it is possible that *during* the execution of a line in the main thread, the underlying layer receives incoming messages and sends the interrupt. The message handler will process these messages *after* the completion of that particular line of code due to the assumption of atomic execution. The only exception is the call to `mac-broadcast`. Messages can still be received and processed when a broadcaster is waiting for the acknowledgement from the abstract MAC layer.

3 COMPUTABILITY

It is important to understand the computability in a given model. Newport and Robinson [10] present randomized consensus algorithms in the abstract MAC layer, and note that they needed to devise new algorithms because of the lack of linearizable shared objects. They argue that because nodes do not know the network size n and cannot collect information from a quorum of nodes, standard register simulation in the standard message-passing model like ABD [2] is "thwarted." We fill the gap by presenting how to simulate linearizable shared objects.

Out Insight: The main reason that ABD requires the communication with a quorum of nodes, because that is the *only* way a node can ensure that its information is *durable* and *timely* in the standard message-passing model. Durable information means that the information is not lost, even after node crashes. Timely information means that the information satisfies the real-time constraint, i.e., after the communication with a quorum is complete, others can learn the information by contacting a quorum of nodes. Our important insight is that the `mac-broadcast` primitive in fact achieves both goals upon learning the acknowledgement. In other words, after the broadcaster learns that the broadcast is complete, it can infer that the message is both durable and timely. It turns out these two properties are essential in satisfying the regularity of the store-collect object.

Store-Collect Object: A store-collect object [3, 4] provides two operations at node i: (i) $Store_i(v)$: store the value v into the object; and (ii) $Collect_i$: collect the set of "most recent" value for each node from the object. The returned value of $Collect_i$ is a *view* – a set of (v_j, j) tuples where j is a node identity, and v_j is its most recent stored value. For each j, there is at most one tuple of $(*, j)$ in V. For each view V, $V(j) = v_j$ if $(v_j, j) \in V$; otherwise, $V(j) = \perp$.

On a high-level, a store-collect object guarantees that each collect operation returns a view containing the most recent value stored by each node; however, it does *not* require the store and collect operations to appear to occur instantaneously. In other words, the object is not necessarily linearizable. Hence, an atomic snapshot object is a store-collect object, but not vice versa. To formally define store-collect, we first discuss a useful notion.

A *history* is an execution of the store-collect object, which can be represented using a partially ordered set $(H, <_H)$. Here, H is the set of invocation (inv) and response ($resp$) events of the Store and Collect operations, and $<_H$ is an irreflexive transitive relation that captures the real-time "occur-before" relation of events in H. Formally, for any two events e and f, we say $e <_H f$ if e occurs before f in the execution. For two operations op_1 and op_2, we say that op_1 precedes op_2 if $resp(op_1) <_H inv(op_2)$.

We assume that every value in a store operation is unique (a condition that can be achieved using sequence numbers and node identifiers). Moreover, a node can have at most one pending operation. Given two views V_1 and V_2 returned by two collect operations, we denote $V_1 \preceq V2$, if for every $(v_1, j) \in V_1$, there exists a v_2 such that (i) $(v_2, j) \in V_2$; and (ii) either $v_1 = v_2$ or the invocation of $Store_j(v_2)$ occurs after the response of $Store_j(v_1)$. That is, from the perspective of node i, v_2 is more recent than v_1. We then say that a history σ satisfies *regularity* if:

- For each Collect $c \in \sigma$ that returns V and for each node j, (i) if $V(j) = \perp$, then no Store by j precedes c in σ; and (ii) if $V(j) = v$, then $Store_j(v)$'s invocation precedes c's response, and there does not exist $Store_j(v')$ such that $v' \neq v$, and $Store_j(v')$'s response occurs after $Store_j(v)$'s response and before c's invocation.

- Consider any pair of two collects in σ, c_1 and c_2, which return views V_1 and V_2, respectively. If c_1 precedes c_2, then $V_1 \preceq V_2$.

Definition 1 (Store-Collect)**.** An algorithm correctly implements the store-collect object if every execution of the algorithm results into a history that satisfies regularity.

Our Construction: To achieve regularity, it is obvious that each stored value has to be durable and timely. Moreover, the collected information by a node needs to be known by subsequent Collect's. These observations together with the aforementioned insight of the `mac-broadcast` primitive give us a straightforward construction of the store-collect object presented below. With a slight abuse of terminology, we use $C = A \cup B$ to denote the merge operation of two views A and B such that the new view C contains the newer value from each node. Recall that each node can have at most one pending operation and each value is unique; hence, the notion of a newer value is well-defined.

Algorithm 1 MAC-SC: Steps at each node i

Local Variable:

$view_i$ ▷view, initialized to $\emptyset$

When $Store_i(v)$ **is invoked**:

1: $currentView_i \leftarrow view_i \cup \{(v, i)\}$
2: mac-broadcast(STORE, $currentView_i$)
3: **return** ACK ▷Store is completed

When $Collect_i$ **is invoked**:

4: $currentView \leftarrow view_i$
5: mac-broadcast(STORE, $currentView$)
6: **return** $currentView$

// *Background message handler*

7: **Upon** receive(STORE, $view$) **do**
8: $view_i \leftarrow view_i \cup view$

Theorem 1. *MAC-SC correctly implements the store-collect object.*

Proof Sketch. ***Property I***: Consider a Collect operation c by node i that returns V. For each node j, consider two cases:

- $V(j) = \perp$: node i has not received any message from j's `mac-broadcast`, so either `mac-broadcast` by j is not yet completed, or node j has not invoked any `mac-broadcast`. In both cases, there is no Store by j that precedes c.
- $V(j) = v$: by construction, v is in $V(j)$ because $Store_j(v)$ is invoked before c completes. Next, we can prove by contradiction that there is no other Store by node j that completes between two events: the response event of $Store_j(v)$ and the invocation of c.

Property II: Suppose c_1 and c_2 are two Collect's such that c_1 returns view V_1, c_2 returns view V_2, and c_1 precedes c_2. When `mac-broadcast` completes, all the fault-free nodes have received the broadcast message. Therefore, $V_1 \preceq V_2$ □

From Store-Collect to Other Shared Objects: The constructions of several well-known objects over the store-collect object are presented in [4]. These constructions only use Store and Collect without relying on other assumptions. Hence, stacking these constructions on top of MAC-SC gives us the ability to simulate many useful objects such as counters, sets, atomic snapshots, and hence, multi-writer atomic registers. This stacked approach bridges the gap of computability between the abstract MAC layer and the standard message-passing model.

4 SUMMARY

Our results demonstrate the power of the abstract MAC layer. Our full paper also presents anonymous storage-efficient approximate and randomized consensus algorithms. For approximate consensus, we introduce a simple "jump" mechanism so that nodes can jump to a higher phase, which leads to reduced storage space. For the randomized consensus, we integrate the framework of alternating adopt-commit objects and conciliator objects [1] with our jump mechanism. These algorithms only need to use a constant number of variables (or values). Our consensus algorithms are *anonymous* in the sense that nodes do not need to have an identity. Prior algorithms developed for the abstract MAC layer and algorithms using the "stacked approach" (which builds on top of our store-collect object) are not anonymous, and require storage space linear with respect to the number of nodes.

REFERENCES

[1] J. Aspnes. A modular approach to shared-memory consensus, with applications to the probabilistic-write model. *Distributed Comput.*, 25(2):179–188, 2012.
[2] H. Attiya, A. Bar-Noy, and D. Dolev. Sharing memory robustly in message-passing systems. *J. ACM*, 42(1):124–142, 1995.
[3] H. Attiya, A. Fouren, and E. Gafni. An adaptive collect algorithm with applications. *Distributed Comput.*, 15(2):87–96, 2002.
[4] H. Attiya, S. Kumari, A. Somani, and J. L. Welch. Store-collect in the presence of continuous churn with application to snapshots and lattice agreement. In S. Devismes and N. Mittal, editors, *Stabilization, Safety, and Security of Distributed Systems - 22nd International Symposium, SSS 2020, Austin, TX, USA, November 18-21, 2020, Proceedings*, volume 12514 of *Lecture Notes in Computer Science*, pages 1–15. Springer, 2020.
[5] H. Attiya and J. Welch. *Distributed Computing: Fundamentals, Simulations, and Advanced Topics.* Wiley Series on Parallel and Distributed Computing, 2004.
[6] M. Biely, P. Robinson, and U. Schmid. Agreement in directed dynamic networks. In G. Even and M. M. Halldórsson, editors, *Structural Information and Communication Complexity*, pages 73–84, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
[7] F. Kuhn, N. A. Lynch, and C. C. Newport. The abstract MAC layer. In I. Keidar, editor, *Distributed Computing, 23rd International Symposium, DISC 2009, Elche, Spain, September 23-25, 2009. Proceedings*, volume 5805 of *Lecture Notes in Computer Science*, pages 48–62. Springer, 2009.
[8] F. Kuhn, N. A. Lynch, and C. C. Newport. The abstract MAC layer. *Distributed Comput.*, 24(3-4):187–206, 2011.
[9] N. A. Lynch. *Distributed Algorithms.* Morgan Kaufmann, 1996.
[10] C. Newport and P. Robinson. Fault-tolerant consensus with an abstract MAC layer. In U. Schmid and J. Widder, editors, *32nd International Symposium on Distributed Computing, DISC 2018, New Orleans, LA, USA, October 15-19, 2018*, volume 121 of *LIPIcs*, pages 38:1–38:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.
[11] C. C. Newport. Consensus with an abstract MAC layer. In M. M. Halldórsson and S. Dolev, editors, *ACM Symposium on Principles of Distributed Computing, PODC '14, Paris, France, July 15-18, 2014*, pages 66–75. ACM, 2014.
[12] D. Yu, Y. Zou, J. Yu, Y. Zhang, F. Li, X. Cheng, F. Dressler, and F. C. M. Lau. Implementing the abstract MAC layer in dynamic networks. *IEEE Trans. Mob. Comput.*, 20(5):1832–1845, 2021.
[13] D. Yu, Y. Zou, Y. Zhang, H. Sheng, W. Lv, and X. Cheng. An exact implementation of the abstract MAC layer via carrier sensing in dynamic networks. *IEEE/ACM Trans. Netw.*, 29(3):994–1007, 2021.